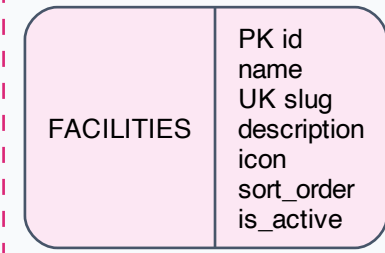


Membership, Classes, and Attendance



Identity and Access

```

    erDiagram
        PERMISSIONS ||--o{ ROLES : "has"
        PERMISSIONS ||--o{ MODEL_HAS_PERMISSIONS : "has"
        ROLES ||--o{ MODEL_HAS_ROLES : "has"
        MODEL_HAS_PERMISSIONS ||--o{ MODEL_HAS_ROLES : "has"
        MODEL_HAS_PERMISSIONS ||--o{ TRAINERS : "has"
        MODEL_HAS_ROLES ||--o{ TRAINERS : "has"
        TRAINERS ||--o{ MEMBERS : "has"
        MEMBERS ||--o{ USERS : "has"
        MEMBERS ||--o{ PERMISSIONS : "has"

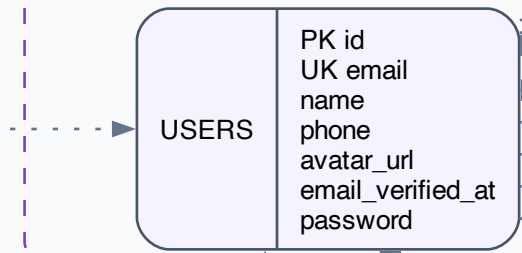
        PERMISSIONS {
            string PK_id PK
            string UK_name_guard_name UK
            string guard_name
        }
        ROLES {
            string PK_id PK
            string UK_name_guard_name UK
            string guard_name
        }
        MODEL_HAS_PERMISSIONS {
            string PK_FK_permission_id PK
            string PK_FK_role_id PK
            string PK_model_type PK
            string PK_model_id PK
        }
        MODEL_HAS_ROLES {
            string PK_FK_role_id PK
            string PK_FK_model_type PK
            string PK_model_id PK
        }
        TRAINERS {
            string PK_id PK
            string FK_UK_user_id FK
            string specialization
            bool is_active
        }
        MEMBERS {
            string PK_id PK
            string FK_UK_user_id FK
            string UK_member_code
            string joined_at
        }
        USERS {
            string PK_id PK
            string UK_email_name_phone_avatar_url_email_verified_at_password UK
            string email_name
            string phone
            string avatar_url
            string email_verified_at
            string password
        }
  
```

The diagram illustrates the following tables and their attributes:

- PERMISSIONS**: PK id, UK name + guard_name, guard_name
- ROLES**: PK id, UK name + guard_name, guard_name
- MODEL_HAS_PERMISSIONS**: PK, FK permission_id, PK model_type, PK model_id
- MODEL_HAS_ROLES**: PK, FK role_id, PK model_type, PK model_id
- TRAINERS**: PK id, FK, UK user_id, specialization, is_active
- MEMBERS**: PK id, FK, UK user_id, UK member_code, joined_at
- USERS**: PK id, UK email, name, phone, avatar_url, email_verified_at, password

Key relationships and cardinalities:

- PERMISSIONS** to **ROLES**: 1 to many
- PERMISSIONS** to **MODEL_HAS_PERMISSIONS**: 1 to many
- ROLES** to **MODEL_HAS_ROLES**: 1 to many
- MODEL_HAS_PERMISSIONS** to **MODEL_HAS_ROLES**: 1 to many
- MODEL_HAS_PERMISSIONS** to **TRAINERS**: 1 to 0..1
- MODEL_HAS_ROLES** to **TRAINERS**: 1 to 0..1
- TRAINERS** to **MEMBERS**: 1 to many
- MEMBERS** to **USERS**: 1 to 0..1
- MEMBERS** to **PERMISSIONS**: 1 to many



Authentication, Notifications, and Audit

```
graph TD
    ACTIVITY_LOG[ACTIVITY_LOG]
    MEDIA[MEDIA]
    DEVICE_TOKENS[DEVICE_TOKENS]
    NOTIFICATIONS[NOTIFICATIONS]
    PERSONAL_ACCESS_TOKENS[PERSONAL_ACCESS_TOKENS]

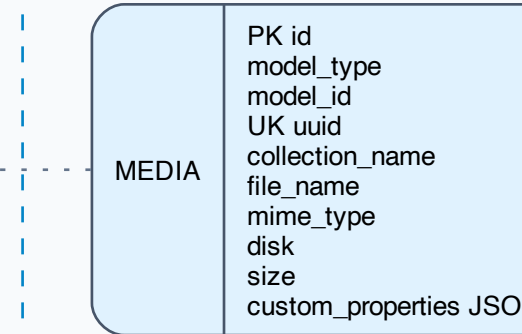
    ACTIVITY_LOG -.-> MEDIA
    ACTIVITY_LOG -.-> NOTIFICATIONS
    ACTIVITY_LOG -.-> PERSONAL_ACCESS_TOKENS
    MEDIA -.-> PERSONAL_ACCESS_TOKENS
    NOTIFICATIONS -.-> PERSONAL_ACCESS_TOKENS
    PERSONAL_ACCESS_TOKENS -.-> DEVICE_TOKENS
```

The diagram illustrates the relationships between several database tables. The tables are represented as rounded rectangles, each divided into two sections: the left section contains the table name, and the right section contains its columns. The columns are listed as follows:

- ACTIVITY_LOG**: PK id, log_name, description, event, subject_type, subject_id, causer_type, causer_id, attribute_changes JSON, properties JSON
- MEDIA**: PK id, model_type, model_id, UK uuid, collection_name, file_name, mime_type, disk, size, custom_properties JSON
- DEVICE_TOKENS**: PK id, FK user_id, UK token, platform
- NOTIFICATIONS**: PK id, UUID, type, notifiable_type, notifiable_id, data JSON, read_at
- PERSONAL_ACCESS_TOKENS**: PK id, tokenable_type, tokenable_id, name, UK token, abilities, last_used_at, expires_at

Relationships are indicated by dashed lines with labels:

- A dashed line connects **ACTIVITY_LOG** to **MEDIA**.
- A dashed line connects **ACTIVITY_LOG** to **NOTIFICATIONS**, labeled "notifiable".
- A dashed line connects **ACTIVITY_LOG** to **PERSONAL_ACCESS_TOKENS**, labeled "tokenable".
- A dashed line connects **MEDIA** to **PERSONAL_ACCESS_TOKENS**.
- A dashed line connects **NOTIFICATIONS** to **PERSONAL_ACCESS_TOKENS**.
- A dashed line connects **PERSONAL_ACCESS_TOKENS** to **DEVICE_TOKENS**.



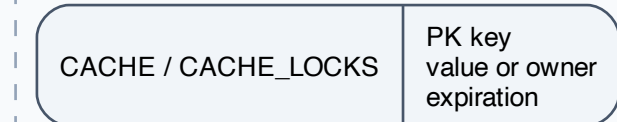
Laravel Infrastructure

The diagram illustrates the relationships between various Laravel infrastructure components. It features four main boxes representing different data stores or tables, connected by dashed lines indicating relationships.

- QUEUE TABLES**: Contains fields: jobs, job_batches, failed_jobs.
- CACHE / CACHE_LOCKS**: Contains fields: PK key, value or owner, expiration.
- SESSIONS**: Contains fields: PK id, user_id logical, ip_address, user_agent, payload, last_activity.
- PASSWORD_RESET_TOKENS**: Contains fields: PK email, token, created_at.

Relationships are indicated by dashed lines:

- A solid line connects **QUEUE TABLES** to **SESSIONS**, labeled `logical_user_id`.
- A solid line connects **SESSIONS** to **PASSWORD_RESET_TOKENS**, labeled `logical_email`.
- A dashed line connects **CACHE / CACHE_LOCKS** to **PASSWORD_RESET_TOKENS**.



Store and Checkout

```
erDiagram
    PRODUCT_CATEGORIES ||--o{ PRODUCTS : "1 to many"
    PRODUCTS ||--o{ ORDER_ITEMS : "1 to many"
    ORDERS ||--o{ ORDER_ITEMS : "1 to many"
    ORDERS ||--o{ PRODUCTS : ""

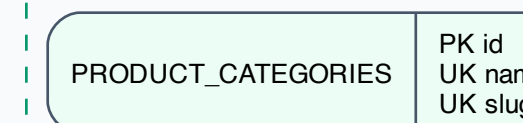
    PRODUCT_CATEGORIES {
        string PK_id PK
        string UK_name UK
        string UK_slug UK
    }
    PRODUCTS {
        string PK_id PK
        string FK_product_category_id FK
        string name
        string description
        float price
        int stock
        string image_url
        bool is_active
    }
    ORDERS {
        string PK_id PK
        string FK_member_id FK
        string status
        string payment_method
        float total_price
        string payment_reference
    }
    ORDER_ITEMS {
        string PK_id PK
        string FK_order_id FK
        string FK_product_id FK
        float quantity
        float unit_price
        float subtotal
    }
```

The diagram illustrates the database structure for a 'Store and Checkout' system. It features four tables: **PRODUCT_CATEGORIES**, **PRODUCTS**, **ORDERS**, and **ORDER_ITEMS**.

- PRODUCT_CATEGORIES** (Primary Key: id, Unique Keys: name, slug) is linked to **PRODUCTS** via a 1 to many relationship.
- PRODUCTS** (Primary Key: id, Foreign Key: product_category_id) is linked to **ORDER_ITEMS** via a 1 to many relationship.
- ORDERS** (Primary Key: id, Foreign Key: member_id) is linked to **ORDER_ITEMS** via a 1 to many relationship.
- ORDERS** is also linked to **PRODUCTS** via a 1 to many relationship.

Table details:

- PRODUCT_CATEGORIES**: id (PK), name (UK), slug (UK)
- PRODUCTS**: id (PK), product_category_id (FK), name, description, price, stock, image_url, is_active
- ORDERS**: id (PK), member_id (FK), status, payment_method, total_price, payment_reference
- ORDER_ITEMS**: id (PK), order_id (FK), product_id (FK), quantity, unit_price, subtotal



polymorphic model